Agent Runtime / Harness Engineering 横纵分析报告

> 从 AutoGPT 的"while True + LLM call"到 OpenAI Harness 和 Martin Fowler 把 Harness Engineering 定义为一个正式工程领域——Agent 基础设施层如何在三年内从一个被忽略的"胶水代码"变成了决定 Agent 成败的关键战场，又如何让行业得出了一个反直觉的结论：**模型可能没那么重要，Harness 才重要。**

一、一句话定义

Agent Runtime / Harness 是让 AI Agent **安全、可靠、可观测地运行**的基础设施层。它包括沙箱隔离、编排引擎、状态管理、护栏、监控和权限控制。如果把 Agent 比作一辆自动驾驶汽车，Model 是发动机，Framework 是车身，**Harness 就是刹车、方向盘和安全带**——没有它，发动机再强也上不了路。

2026 年 4 月，OpenAI 把 Harness 从 Codex 的内部组件上升为 Agents SDK 的一等公民，Martin Fowler 发表《Harness Engineering for Coding Agent Users》将它定义为正式工程领域。一个共识正在形成：**GPT-4.5 在一个优秀的 Harness 里，比 GPT-5.5 在一个简陋的 Harness 里表现更好。**

> 🎯 **读完这篇你能**：理解沙箱隔离、编排引擎、护栏监控四层 Harness 架构，能判断一个 Agent 应用在上线前还缺哪些安全与可靠性基础设施。

二、技术背景

> 前置知识请参考独立文档 《Agent Runtime / Harness 前置知识》，涵盖沙箱、编排引擎、Guardrails、MCP Gateway、可观测性等概念的通俗讲解。

三层架构

Martin Fowler 和 LangChain 的 Harrison Chase 在 2026 年独立提出了同一套三层模型：

```
Agent Framework（编程范式：LangGraph / CrewAI / Dify）
        ↕
Agent Runtime（执行引擎：状态管理 / 持久化 / 扩缩容）
        ↕
Agent Harness（控制层：沙箱 / 护栏 / 权限 / 监控 / 人机协作接口）
```

Framework 管"怎么写出 Agent"，Runtime 管"怎么跑起来 Agent"，Harness 管"怎么安全地跑 Agent"。

为什么 Harness 突然变得重要?

一个简单的对比就能说清楚:

	2023 年的 Agent	2026 年的 Agent
运行方式	单进程 Python 脚本	容器隔离 + 持久化状态
安全	无	沙箱 + Guardrails + 权限审计
失败处理	挂了就白跑	Durable execution + 自动重试
可观测性	print 调试	OpenTelemetry + 全链路追踪
多 Agent 协作	手动串行	图编排 + MCP/A2A

2023 年的问题不是"Agent 不够聪明"——问题是 Agent 不可靠、不安全、不可观测。Harness 解决的就是这三件事。

> 📚 **关联报告**
>
> – AI Agent — Runtime/Harness 是 Agent 的运行时基础设施，Agent 报告讲"是什么"，本篇讲"怎么跑起来"
>
> – LLM 应用框架 — LangGraph、Dify 等框架的编排层与 Harness 有重叠
>
> – AI Safety — 护栏（Guardrails）是 Harness 的安全层

三、纵向分析：从"while True"到 Harness 范式

3.1 萌芽期：自主循环的原始冲动（2023）

2023 年 3 月，AutoGPT 发布。 这个由 Significant Gravitas 一个人开发的项目，几周内冲到 10 万+ Stars。它的底层逻辑极其简单——一个永不停止的 for 循环:

```
while True:
    thought = llm.think(context)    # "下一步该做什么？"
    action = llm.act(thought)       # 执行工具调用
    result = execute(action)        # 拿到结果
    context.append(result)          # 喂回上下文
```

这个循环在今天看来粗糙到可笑——没有状态管理、没有错误恢复、没有沙箱、token 消耗无限增长。但它定义了 Agent Runtime 的第一个原型：**Agent 是一个需要持续运行的进程，不是一次性的请求–响应。**

同月，LangChain 0.0.161 发布了 AgentExecutor——用 ReAct（Reasoning + Acting）模式把"思考→行动→观察"循环抽象为可编程的框架。相比 AutoGPT，AgentExecutor 最大的进步是**有了一个"Executor"的概念**——Agent 不直接执行自己的决策，而是通过 Executor 这个中间层来执行。这个"中间层"就是后来 Harness 的雏形。

这个阶段的本质是：Agent Runtime = while True + LLM call。 没人认真对待运行时——所有人都盯着"Agent 能做多少事"，没人问"Agent 在做事的过程中崩了怎么办"。

3.2 框架化期：图引擎和编排层（2024）

2024 年，Agent Runtime 从"单循环"走向了"图编排"。

LangGraph 在 2024 年 1 月发布，引入 StateGraph——不只支持链式调用，而是任意图结构的 Agent 工作流。每个节点是一个处理步骤，边表示状态转移。“状态”不再是临时变量，而是一个持久化的 State 对象。

Dify 的 Workflow 引擎走了另一条路——可视化 DAG。非技术人员可以拖拽节点搭建 Agent 工作流。底层基于拓扑排序的图执行引擎，每个节点独立运行。

CrewAI 在 2024 年中引入了多 Agent 编排——Manager-Worker 层次架构。一个 Manager Agent 分解任务、分配工作、协调结果。

2024 年 10 月，Temporal（企业级工作流引擎）发表了"durable multi-agentic AI architecture"——将 AI Agent 运行在 Temporal Workflow 中，实现持久化执行和故障恢复。Agent 在任何步骤中断后都能恢复到中断前的状态继续运行。这解决了"跑了三天的 Agent 挂了就得重来"的痛点。

这个阶段的本质是：编排越来越成熟，但运行时仍然脆弱。 框架画得出漂亮的多 Agent 流程图，但一旦上线运行，debug 还是靠 print。

3.3 生产基建期：云厂商入场 + 沙箱 + 可观测性（2025）

2025 年是 Agent Runtime 历史上的分水岭——**云厂商全面入局，把 Agent 运行时从"开源玩具"变成了"托管服务"。**

LangGraph Platform GA（2025.05） 是开源阵营的回应——StateGraph 持久化、Human-in-the-loop、多 Agent 拓扑管理，让 LangGraph 从一个代码库变成了一个可部署的平台。

AWS Bedrock AgentCore（2025.07） 是第一个云厂商级 Agent 专用运行时。底层基于 Firecracker microVM（AWS Lambda 同款隔离技术），为每个 Agent 会话提供独立的执行环境。1 亿美元 Agent 生态投资。由此，Agent 的"执行环境"和"安全隔离"第一次被当作云平台的一等公民来设计。

Google Vertex AI Agent Engine（2025） 和 **Azure AI Foundry Agent Service（2025.05 GA）** 同步跟进。三家云厂商在同一年推出了自己的 Agent 运行时方案——标准打架是后话，但"Agent 需要专用运行时"这个事实已经被行业承认。

LangSmith + OpenTelemetry 集成（2025） 让 Agent 可观测性有了标准——不只是"LLM 调用了哪些 API"，而是全链路追踪：prompt → LLM call → tool call → state change。

Guardrails 也成熟了：NVIDIA NeMo Guardrails 发布 NIM 微服务版本（话题限制 + 内容安全 + 越狱检测），Meta 开源 LlamaFirewall，OpenGuardrails 统一方案出现。

MCP 生态在企业级落地：2025 年 12 月 Anthropic 将 MCP 捐给 Linux 基金会，成立 Agentic AI Foundation。MCP Gateway 成为新热点——Solo.io、Traefik、Harmonic Security 等公司相继推出 MCP 网关产品。

沙箱技术定型：gVisor（Google/用户态内核）、Kata Containers（轻量级 VM/每个 Agent 独立内核）、Firecracker（AWS/microVM）三条路线各有定位。

这个阶段的本质是：Agent Runtime 从"框架的一个模块"升级为"独立的云服务品类"。 云厂商入局意味着这个品类被正式承认为一个独立的市场。

3.4 Harness 范式确立（2026）

2026 年 1 月到 4 月，是 Agent Runtime 从"基建"升维为"工程范式"的关键窗口。

1 月，OpenAI 发表《Unrolling the Codex Agent Loop》——第一次向外部公开了 Codex CLI 的 Agent Loop 机制：规划→执行→观察循环、上下文管理、错误恢复。

2 月，《Unlocking the Codex Harness》 正式定义了 Harness 的位置：Agent 与底层模型之间的中间层，负责会话管理、权限控制、工具路由、沙箱管理、状态持久化。这不是框架，不是运行时，而是一个**控制面（Control Plane）**。

4 月 15 日，Agents SDK v0.14.0 把 Harness 从 Codex 专有组件变成了开放 API——Model-Native Harness、沙箱执行、长时间运行 Agent 支持。OpenAI 说："沙箱不是 optional 的安全功能，它是 Agent 正常运行的前提。"

4 月 28 日，Symphony 开源——Codex 编排层的开放标准规范，可管理多 Agent 协作工作流。OpenAI 内部使用效果：PR 数量增长 5 倍。

Martin Fowler（ThoughtWorks 首席科学家）在 2026 年发表《Harness Engineering for Coding Agent Users》，正式将 Harness Engineering 定义为：**"设计和管理 AI Coding Agent 运行的工程实践"**。核心关注：上下文窗口管理、工具选择策略、安全边界、错误恢复、人类监督循环。

LangChain 的 Harrison Chase 同期发表《Agent Frameworks, Runtimes, and Harnesses - oh my!》——Framework/Runtime/Harness 的三层划分正式被行业采纳。

4 月，Red Hat 发布 Tank OS——将 OpenClaw 打包为可启动 Linux 镜像（基于 fedora-bootc），提供 OS 级的 Agent 隔离——Harness 的极端形态。

Anthropic 的入场：Managed Agents（2026.04.10） 是整个 Harness 叙事里最戏剧性的转折。在 OpenAI 定义了 Harness 的话语权后不到两周，Anthropic 抛出了自己的答案——一个全托管的 Agent 云平台。

Claude Managed Agents 的架构核心是 **"解耦大脑与双手"**（Decoupling the brain from the hands）。传统理解里 Agent = LLM + 工具调用，Managed Agents 把这条链路拆成三层：

- **Agent**（大脑）：定义目标、规划步骤、决策分支，管"想什么"
- **Environment**（双手）：内置沙箱，提供文件系统、终端、浏览器、Git 操作能力，管"做什么"
- **Session**（执行管道）：持久化状态、自动重试、审计日志，管"怎么持续跑"

这与 OpenAI Agents SDK 的根本差异不在功能清单，而在**设计哲学**：OpenAI 把 Harness 视为模型的"附属控制面"，Anthropic 把 Harness 视为模型的"独立执行层"——Environment 与模型之间通过 API 边界解耦。好处是 Agent 崩溃不影响 Environment 状态，代价是多了跨进程通信的开销。

Vault（凭证保险库） 是 Anthropic 在 Harness 安全上的独特创新——Agent 调用外部 API 时不直接接触 token，所有凭证存在 Vault 中，Agent 只拿一个临时授权句柄。这个设计回答了 Harness 安全的核心矛盾：Agent 既要能自主调用外部服务，又不能持有敏感凭证。

并行 Agent 团队：Managed Agents 支持多个子 Agent 同时执行独立子任务，主 Agent 负责协调和汇总。Anthropic 官方给出的实验数据是"完成率和稳定性显著提高"。

发布后行业反应两极分化。支持者认为这是"Agent 从概念到生产的 10 倍加速器"（将数月的 Agent 基建压缩到几天）。批评者指出了**厂商锁定风险**：Managed Agents 的 Environment 层与 Anthropic 云深度绑定，Agent 离开这个环境无法运行——这跟 OpenAI 把 Harness 做进 SDK 的开放策略形成了鲜明对比。

海通国际研报明确评价"头部大模型公司入局 Agent Infra，标志 AI 行业向系统层深化"。在此之前，Agent Infra 是云厂商和创业公司的地盘，OpenAI 和 Anthropic 的入场意味着 Harness 不再是谁都能做的"胶水代码"，而是模型公司手里的一张牌。

Microsoft Foundry Hosted Agents（2026.04 公测） 几乎跟 Anthropic 同期发布。核心卖点是"零成本闲置"（Agent 不运行时不计费）、企业身份治理（Entra ID 集成、Role-based Access Control）和持久状态（Agent 中断后从断点恢复）。三条差异化线索：比 Anthropic 更"企业合規导向"，比 AWS/Google 更"Azure 生态绑定"，比自建更"开箱即用"。

其他值得关注的变化：

- **Cloudflare Agent Cloud**（2026）正式入局，利用全球边缘节点为 Agent 提供低延迟执行环境
- **Google Gemini Enterprise Agent Platform**（2026）把 Agent Harness 与管理控制台、IAM 权限、审计日志打通，主打企业合规
- **Cursor 3.2**（2026.04）创造性地把自己定位为"Agent Execution Runtime"——IDE 不只是写代码的工具，也是 Agent 运行的环境
- **OpenAI + AWS 合作**：AWS Bedrock 上部署 OpenAI 的 Stateful Agent Runtime，两个竞争对手在 Harness 层的互利合作

一个反直觉的共识在 2026 年成形："The Model Doesn't Matter. The Harness Does." OpenAI 内部的"Extreme Harness Engineering"实验（3 人团队、5 个月、0 人类代码，靠 Agent 协作构建生产系统）和海通国际的研报《Big Model vs Big Harness》都指向同一结论：**当模型能力趋同时，Harness 才是决定 Agent 实际表现的关键变量。**

四、横向分析：2026 年 5 月 Agent Runtime 竞争图谱

4.1 核心产品对比

产品	发布方	发布时间	核心定位	沙箱技术	MCP 支持	编排方式	开源
Claude Managed Agents	Anthropic	2026.04	全托管 Agent 云平台	内置沙箱（文件/终端/浏览器）	✅	Agent→Environment→Session	❌
OpenAI Agents SDK + Harness	OpenAI	2025.03→2026.04	模型原生控制面	API层沙箱	部分	Handoff + Symphony	❌
Microsoft Foundry Hosted Agents	Microsoft	2026.04 公测	企业合规 Agent 容器	专用容器 + Entra ID	✅	托管编排 + 持久状态	❌
AWS Bedrock AgentCore	AWS	2025.07	云托管 Agent 运行时	Firecracker microVM	✅	多框架兼容	❌
Gemini Enterprise Agent Platform	Google	2026	企业级 Agent 平台	gVisor (GKE)	✅	IAM + 审计集成	❌
Vertex AI Agent Engine	Google	2025	GCP Agent 运行时	gVisor (GKE)	✅	ADK 5种模式	❌
LangGraph Cloud	LangChain	2025.05	有状态 Agent 编排平台	依赖外部	✅	StateGraph	❌
Cloudflare Agent Cloud	Cloudflare	2026	边缘 Agent 运行时	边缘隔离	✅	全球边缘节点	❌
Dify Workflow	LangGenius	2024	可视化 Agent 编排	依赖外部	✅	拖拽 DAG	✅
Tank OS	Red Hat	2026.04	OS级 Agent 隔离	系统级容器	✅	不可变镜像	✅
Temporal + Agents	Temporal	2024.10	Durable Execution	依赖外部	发展中	Workflow	✅

4.2 沙箱技术对比

方案	开发者	隔离级别	启动速度	资源开销	适用场景
Firecracker	AWS	microVM（独立内核）	<125ms	极低	会话级隔离，高密度部署
gVisor	Google	用户态内核	~200ms	低	K8s 容器化 Agent
Kata Containers	社区（Intel/蚂蚁）	轻量级 VM	~500ms	中	高安全需求 Agent
API 级沙箱	OpenAI	权限控制 + 网络隔离	~0ms	零	代码执行，工具调用

4.3 Guardrails（护栏）对比

方案	开发者	发布时间	功能覆盖	部署方式	开源
NeMo Guardrails	NVIDIA	2025	话题/内容/越狱	NIM 微服务	✅
LlamaFirewall	Meta	2025.05	输入/输出过滤	开源库	✅
Guardrails AI	社区	2023→	结构化验证	Python 库	✅
OpenAI Guardrails	OpenAI	2026	API 集成	API 内置	❌

4.4 MCP 网关对比

方案	发布时间	核心能力
Solo.io Agent Gateway	2025.08	已贡献给 LF，AI Gateway 标准
Traefik MCP Gateway	2025	+NVIDIA Safety NIMs 集成
Harmonic Security Gateway	2025.10	可视化 MCP 流量、治理 Shadow MCP
Diagrid MCP Gateway	2025	安全治理、速率限制

4.5 可观测性方案

方案	类型	核心能力
LangSmith + OTel	商业	Agent 全链路追踪，LLM call → tool call → state
OpenTelemetry GenAI	开源标准	LLM/tool/agent 调用的标准 trace 格式
Cisco AGNTCY	商业	多 Agent 系统可观测性
Datadog LLM Observability	商业	原生支持 OTel GenAI 语义约定

五、横纵交汇洞察

5.1 从"胶水代码"到"基础设施"——Harness 为什么被忽视了三年

回看 2023–2026 的演变，最反直觉的不是技术本身，而是 **Harness 的价值被整个行业忽视了三年**。

2023 年所有人都在问"Agent 能做多少事"（AutoGPT 的 Demo 有多惊艳），2024 年问"怎么编排多个 Agent"（LangGraph/CrewAI 的热度），2025 年问"云厂商能托管 Agent 了吗"（Bedrock/Vertex AI/Foundry 的发布）。直到 2026 年才有人问："**Agent 跑崩了多少次？安全问题解决了吗？**"

这个认知延迟的根本原因是：**Agent 从 Demo 到生产走了比预期更长的路。** Demo 阶段，Agent 的好坏仅取决于模型能力——GPT–4 > GPT–3.5。但生产阶段，Agent 的好坏取决于运行时质量——GPT–4.5 + 优秀 Harness > GPT–5.5 + 简陋 Harness。Harness 的价值是在生产环境里被"打"出来的——删库事件、token 耗尽、安全攻击，每一次事故都在提醒行业：没有护栏的 Agent 是危险的。

5.2 "The Model Doesn't Matter" 是一半真相

"模型不重要"这个判断在 2026 年很流行，但它只说对了一半。

对的那一半是：当模型能力进入边际递减期（GPT–5.5 比 GPT–5.4 的提升远小于 GPT–5 比 GPT–4.5 的提升），**Harness 的边际收益开始超过模型的边际收益**。OpenAI 内部实验确认了这一点。

错的那一半是：**如果底层模型不够好，再好的 Harness 也没用。** 2023 年的 GPT–3.5 放在 2026 年最先进的 Harness 里，也不可能做到 SWE–bench 87.6%。Harness 放大模型的能力，但不能创造能力。

更准确的表述是：**模型决定 Agent 的上限，Harness 决定 Agent 的下限。** 上限是"Agent 能做多难的事"，下限是"Agent 会不会把事搞砸"。

5.3 沙箱——Harness 的核心战场

四条沙箱技术路线（Firecracker、gVisor、Kata、API 级）在 2026 年各有胜负。但它们的竞争本质不是技术优劣——而是对"Agent 安全"的定义不同。

- **API 级沙箱**（OpenAI）认为安全 = 权限控制——Agent 只被允许调用白名单 API，网络隔离，资源配额。启动快，零开销。
- **Firecracker**（AWS）认为安全 = 执行隔离——每个 Agent 会话跑在独立的 microVM 里，就算一个 Agent 崩溃也不影响其他人。
- **gVisor**（Google）认为安全 = 内核隔离——Agent 的代码和应用代码跑在不同的"视界"里。
- **Tank OS**（Red Hat）认为安全 = 操作系统隔离——整个 Agent 环境打包成一个不可变的 Linux 镜像，从文件系统到网络到进程全部隔离。

2026 年还没有标准答案。但趋势很清晰：**Agent 越自主，需要的隔离层级越高。** 一个只做客服问答的 Agent，API 级沙箱足够。一个能自主编程、操作文件系统的 Agent，至少需要 container 级隔离。一个能控制操作系统、安装软件的 Agent，需要 VM 级隔离。

5.4 未来推演：三个剧本

剧本一（最可能）：Harness 成为独立的云服务品类（概率 60%）

Agent Runtime 从云厂商的"附加功能"升级为独立的云服务产品线——类似今天的"数据库即服务"或"容器即服务"。Harness 的三大组件（沙箱、护栏、可观测性）被标准化。ML 工程师的 job description 里会出现"Harness Engineering"。

剧本二（最危险）：Harness 碎片化导致 Agent 安全灾难（概率 25%）

每家云厂商、每个框架都有自己的 Harness 标准，互不兼容。MCP 和 A2A 在协议层统一了部分，但在运行时层完全碎片化。一次大规模 Agent 安全事件（比如 Agent 被 Prompt Injection 攻击后自动执行了危险操作）成为"Agent 的切尔诺贝利"，触发监管收紧。

剧本三（最乐观）：Harness 成为"AI 时代的操作系统"（概率 15%）

如果 Agent 成为人与数字世界的交互接口，那 Harness 就是 Agent 的操作系统层级——管理资源、隔离进程、提供安全、调度任务。OpenAI 的 Symphony、Google 的 Agent Engine、Red Hat 的 Tank OS 都在指向同一个方向。未来 LLM 是 CPU，Agent 是 App，Harness 是 OS。

六、信息来源

核心论文与文章

论文/文章	来源	访问时间
"The Model Doesn't Matter, The Harness Does"	dev.to (2026)	2026-04-30
"Big Model vs Big Harness"	海通国际研报 (2026.04)	2026-04-30
Martin Fowler: "Harness Engineering for Coding Agent Users"	martinfowler.com (2026)	2026-04-30
LangChain: "Agent Frameworks, Runtimes, and Harnesses"	langchain.com (2026)	2026-04-30
沙箱技术对比	segmentfault.com/a/1190000047288554	2026-04-30
Anthropic: "Scaling Managed Agents: Decoupling the brain from the hands"	anthropic.com/engineering/managed-agents (2026.04)	2026-05-01
"Claude 4.6 托管代理详解"	developer.aliyun.com/article/1727149	2026-05-01
Anthropic: "Claude Managed Agents: get to production 10x faster"	claude.com/blog/claude-managed-agents (2026.04)	2026-05-01

产品与技术来源

来源	URL	访问时间
Anthropic Claude Managed Agents	platform.claude.com/docs/en/managed-agents (2026.04)	2026-05-01
Microsoft Foundry Hosted Agents	devblogs.microsoft.com/foundry (2026.04)	2026-05-01
Cloudflare Agent Cloud	cloudflare.com (2026)	2026-05-01
Google Gemini Enterprise Agent Platform	cloud.google.com (2026)	2026-05-01
Cursor 3.2 Agent Runtime	futurumgroup.com (2026.04)	2026-05-01
AutoGPT Architecture	github.com/Significant-Gravitas/AutoGPT	2026-04-30
LangChain AgentExecutor	docs.langchain.com (2023)	2026-04-30
LangGraph Platform GA	changelog.langchain.com (2025.05)	2026-04-30
Dify Workflow	github.com/langgenius/dify	2026-04-30
CrewAI Flows	learn.deeplearning.ai	2026-04-30
Temporal + AI Agents	temporal.io/blog (2024.10)	2026-04-30
AWS Bedrock AgentCore	venturebeat.com (2025.07)	2026-04-30
Vertex AI Agent Engine	cloud.google.com (2025)	2026-04-30
Azure AI Foundry Agent Service	infoq.com (2025.05)	2026-04-30
OpenAI: "Unrolling the Codex Agent Loop"	openai.com (2026.01)	2026-04-30
OpenAI: "Unlocking the Codex Harness"	openai.com (2026.02)	2026-04-30
OpenAI Agents SDK v0.14.0	openai.com (2026.04.15)	2026-04-30
OpenAI Symphony	openai.com (2026.04.28)	2026-04-30
Red Hat Tank OS	techcrunch.com (2026.04)	2026-04-30
NVIDIA NeMo Guardrails	blogs.nvidia.com (2025)	2026-04-30
Meta LlamaFirewall	infoq.com (2025.05)	2026-04-30
MCP → Linux Foundation	anthropic.com (2025.12)	2026-04-30
LangSmith + OpenTelemetry	langchain.com (2025)	2026-04-30
Cisco AGNTCY	outshift.cisco.com (2025)	2026-04-30
Google Agent Sandbox (gVisor)	cloud.google.com (2025.11)	2026-04-30

本文是横纵分析系列的第 26 篇报告。方法论：横纵分析法——纵向沿时间轴追溯 Agent Runtime 从 AutoGPT 到 Anthropic Managed Agents 的四年演变，横向在 2026 年 5 月对比沙箱、护栏、MCP 网关、可观测性方案及全托管 Harness 平台的竞争格局。